

Perfeito! Vou te enviar **todo o conteúdo em formato estruturado** para o LibreOffice, com:

1. **Capítulos e subcapítulos** (para sumário automático)
2. **Tabelas** bem formatadas
3. **Código com fonte monoespaçada**
4. **Diagramas ASCII** (para visualização)
5. **Marcadores e numeração** clara

 DOCUMENTO COMPLETO - VFS DO MEUSO
"Tudo é um Arquivo" - Implementação Completa

Autor: Projeto MeuSO
Data: Agosto 2026
Versão: 1.0
Status: Implementado e Testado

SUMÁRIO

1. [INTRODUÇÃO](#1-introdução)
2. [ARQUITETURA DO SISTEMA](#2-arquitetura-do-sistema)
3. [ESTRUTURAS DE DADOS](#3-estruturas-de-dados)
4. [DRIVERS IMPLEMENTADOS](#4-drivers-implementados)
5. [API DO VFS](#5-api-do-vfs)
6. [SHELL E COMANDOS](#6-shell-e-comandos)
7. [CÓDIGO FONTE COMPLETO](#7-código-fonte-completo)
8. [COMPILAÇÃO E INSTALAÇÃO](#8-compilação-e-instalação)
9. [TESTES E VALIDAÇÃO](#9-testes-e-validação)
10. [REFERÊNCIAS](#10-referências)

1. INTRODUÇÃO

1.1 Objetivo

Implementar o paradigma **"Tudo é um arquivo"** no sistema operacional MeuSO (m68k), permitindo que dispositivos, informações do sistema e arquivos reais sejam acessados através das mesmas funções padronizadas.

1.2 Conceito

No Linux/Unix, tudo pode ser tratado como um arquivo:

Tipo	Exemplo	Descrição
Arquivos de disco	`/boot.bat`	Dados armazenados em FATFS
Dispositivos de hardware	`/dev/tty`	Terminal (vídeo/teclado)
Dispositivos virtuais	`/dev/null`	Descarta dados
Informações do sistema	`/proc/meminfo`	Memória disponível

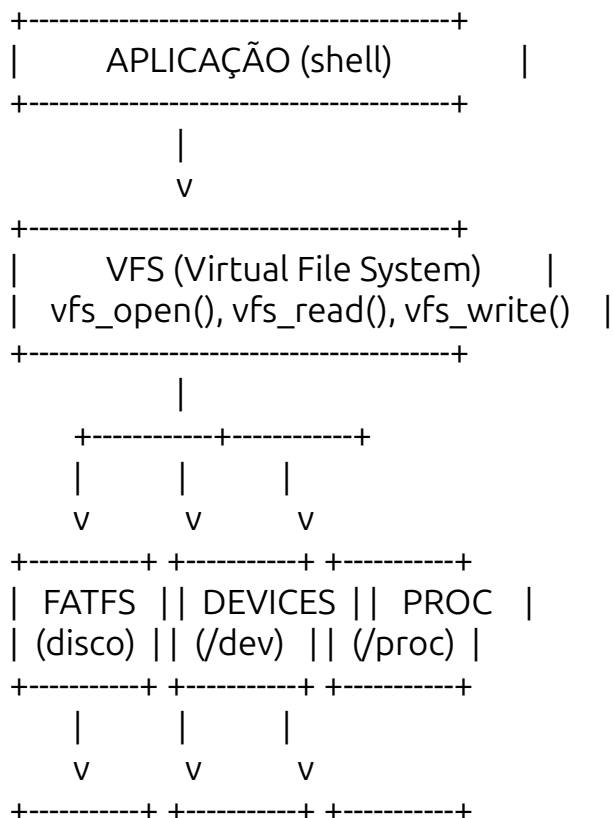
1.3 Benefícios

- **Padronização**: Uma única API para tudo
- **Simplicidade**: Código mais limpo e reutilizável
- **Flexibilidade**: Fácil adicionar novos dispositivos
- **Redirecionamento**: Suporte a `>` e `<` no shell
- **Pipes**: Base para comunicação entre processos (futuro)

2. ARQUITETURA DO SISTEMA

2.1 Visão Geral

...



```
| SD Card || Hardware || Sistema |
| (FATFS) || (UART, || (memória, |
|      || Video) || CPU)   |
+-----+ +-----+ +-----+
...

```

2.2 Camadas do Sistema

Camada 1: Aplicação (Shell)

- Interface com o usuário
- Comandos como `cat`, `echo`, `ls`
- Não sabe se está lendo de arquivo, TTY ou /proc

Camada 2: VFS (Virtual File System)

- Gerencia descritores de arquivos (FD)
- Roteia chamadas para o driver correto
- Mantém tabela de arquivos abertos

Camada 3: Drivers

- ****FATFS****: Arquivos em disco (usa libfileio)
- ****TTY****: Terminal (getchar/putchar)
- ****NULL****: Dispositivo nulo
- ****ZERO****: Gerador de zeros
- ****PROC****: Informações do sistema

Camada 4: Hardware/Sistema

- SD Card (via FATFS)
- UART/Teclado (via getchar)
- Vídeo (via putchar)
- Memória/CPU (para /proc)

3. ESTRUTURAS DE DADOS

3.1 Estrutura File

```
```c

```

```
typedef struct File {
 char *name; // Nome do arquivo/dispositivo
 int type; // FILE_TYPE_REGULAR, DEVICE, PROC
 size_t position; // Posição atual de leitura/escrita
 int ref_count; // Quantos processos usam
 void *private_data; // Dados específicos do driver

 // Operações (polimorfismo)

```

```

int (*open)(struct File *file, const char *path, int flags);
int (*read)(struct File *file, void *buffer, size_t size);
int (*write)(struct File *file, const void *buffer, size_t size);
int (*close)(struct File *file);
int (*ioctl)(struct File *file, int cmd, void *arg);
size_t (*lseek)(struct File *file, size_t offset, int whence);
} File;
```

```

| Campo | Tipo | Descrição |
|----------------|--------------------|---|
| `name` | `char*` | Nome do arquivo (ex: "/boot.bat") |
| `type` | `int` | FILE_TYPE_REGULAR, FILE_TYPE_DEVICE, FILE_TYPE_PROC |
| `position` | `size_t` | Posição atual de leitura/escrita |
| `ref_count` | `int` | Número de processos usando este arquivo |
| `private_data` | `void*` | Dados internos do driver (ex: FIL*) |
| `read` | `function pointer` | Função de leitura do driver |
| `write` | `function pointer` | Função de escrita do driver |
| `close` | `function pointer` | Função de fechamento do driver |

3.2 Estrutura DeviceDriver

```

```c
typedef struct {
 const char *path; // Caminho do dispositivo
 int (*open)(File *, const char *, int);
 int (*read)(File *, void *, size_t);
 int (*write)(File *, const void *, size_t);
 int (*close)(File *);
 int (*ioctl)(File *, int, void *);
 size_t (*lseek)(File *, size_t, int);
} DeviceDriver;
```

```

| Campo | Tipo | Descrição |
|---------|--------------------|---|
| `path` | `const char*` | Caminho do dispositivo (ex: "/dev/tty") |
| `open` | `function pointer` | Abre o dispositivo |
| `read` | `function pointer` | Lê do dispositivo |
| `write` | `function pointer` | Escreve no dispositivo |
| `close` | `function pointer` | Fecha o dispositivo |
| `ioctl` | `function pointer` | Controle do dispositivo |
| `lseek` | `function pointer` | Mova posição (se suportado) |

3.3 Tabela de Descritores

```

```c
static File *fd_table[256]; // Máximo 256 arquivos abertos
static int fd_count = 0; // Quantos estão em uso
```

```

| Descritor | Uso | Dispositivo |
|-----------|------------------------|-------------|
| 0 | stdin (entrada padrão) | /dev/tty |
| 1 | stdout (saída padrão) | /dev/tty |
| 2 | stderr (saída de erro) | /dev/tty |
| 3-255 | Arquivos abertos | Vários |

4. DRIVERS IMPLEMENTADOS

4.1 Driver FATFS (vfs_fat.c)

****Função:**** Acessar arquivos no sistema FATFS

****Dependências:**** `libfileio.a` (suas syscalls)

****Operações:****

| Função | Syscall usada | Descrição |
|---------------|---------------|-----------------------|
| `fat_open()` | `fopen()` | Abre arquivo no disco |
| `fat_read()` | `fread()` | Lê do arquivo |
| `fat_write()` | `fwrite()` | Escreve no arquivo |
| `fat_close()` | `fclose()` | Fecha o arquivo |
| `fat_lseek()` | `fseek()` | Move posição |

****Estrutura privada:****

```

```c
typedef struct {
 FIL *fat_file; // Ponteiro para o FIL (alocado no heap)
 BYTE mode; // Modo de abertura (FA_READ, FA_WRITE, etc.)
 FSIZE_t size; // Tamanho do arquivo (cache)
} FatPrivate;
```

```

****Mapeamento de flags:****

| VFS Flag | FATFS Flag |
|------------|------------|
| `O_RDONLY` | `FA_READ` |
| `O_WRONLY` | `FA_WRITE` |

```
`O_RDWR`	`FA_READ`	`FA_WRITE`
`O_CREAT`	`FA_CREATE_ALWAYS`	
`O_APPEND`	`FA_OPEN_APPEND`	
```

4.2 Driver TTY (vfs_tty.c)

****Função:**** Terminal de texto (vídeo/teclado)

****Dependências:**** `getchar()`, `putchar()` do sistema

****Operações:****

| Função | Descrição |
|---------------|----------------------------------|
| `tty_open()` | Sempre sucesso |
| `tty_read()` | Lê do teclado (getchar) com echo |
| `tty_write()` | Escreve no vídeo (putchar) |
| `tty_close()` | Sempre sucesso |
| `tty_ioctl()` | Comandos especiais (limpar tela) |
| `tty_lseek()` | Não suportado (retorna -1) |

****Comandos ioctl:****

| Comando | Valor | Descrição |
|--------------|-------|--------------------------|
| `TIOC_CLEAR` | 0x01 | Limpa a tela |
| `TIOC_GETXY` | 0x02 | Obtém posição do cursor |
| `TIOC_SETXY` | 0x03 | Define posição do cursor |

****Comportamento especial:****

- Echo automático dos caracteres digitados
- Suporte a backspace (apaga caractere)
- Conversão de `\\n` para `\\r\\n` na escrita

4.3 Driver NULL (vfs_null.c)

****Função:**** Dispositivo nulo (descarta dados)

****Operações:****

| Função | Comportamento |
|---------------|----------------|
| `null_open()` | Sempre sucesso |

```
`null_read()`	Retorna 0 (fim de arquivo)
`null_write()`	Retorna `size` (descarta dados)
`null_close()`	Sempre sucesso
`null_lseek()`	Suporta seek
```

****Uso típico:****

```
```bash
Descartar saída
echo "texto" > /dev/null
```

```
Ler nada (fim de arquivo)
cat /dev/null
```
```

4.4 Driver ZERO (vfs_zero.c)

****Função:**** Gerador de zeros

****Operações:****

```
Função	Comportamento
`zero_open()`	Sempre sucesso
`zero_read()`	Preenche buffer com zeros
`zero_write()`	Retorna `size` (descarta)
`zero_close()`	Sempre sucesso
`zero_lseek()`	Suporta seek
```

****Uso típico:****

```
```bash
Gerar 100 zeros
dd if=/dev/zero of=zeros.bin bs=100 count=1
```
```

4.5 Driver PROC (vfs_proc.c)

****Função:**** Informações do sistema

****Arquivos suportados:****

```
Arquivo	Conteúdo	Função
```

| `/proc/meminfo` | Informações de memória | `proc\_meminfo\_open()` |
| `/proc/cpuinfo` | Informações da CPU | `proc\_cpuinfo\_open()` |
| `/proc/uptime` | Tempo desde o boot | `proc\_uptime\_open()` |

****Estrutura de dados:****

```
```c
typedef struct {
 char *data; // Dados gerados dinamicamente
 size_t size; // Tamanho dos dados
 size_t position; // Posição atual de leitura
} ProcData;
```
```

****Comportamento:****

- Dados são gerados no `open()`
- Leitura é feita dos dados gerados
- Não suporta escrita
- Suporta seek

****Exemplo de saída (/proc/meminfo):****

...

MemTotal: 1024 KB
MemFree: 512 KB
MemUsed: 512 KB
MemAvail: 256 KB
...

****Exemplo de saída (/proc/cpuinfo):****

...

CPU: Motorola 68000
Clock: 8 MHz
Features: FPU (sim)
Cores: 1
Family: m68k
...

5. API DO VFS

5.1 vfs_open()

****Prototype:****

```
```c
int vfs_open(const char *path, int flags);
```
```


****Parâmetros:****

| Parâmetro | Tipo | Descrição |
|-----------|---------------|--|
| `path` | `const char*` | Caminho do arquivo/dispositivo |
| `flags` | `int` | Flags de abertura (O_RDONLY, O_WRONLY, etc.) |

****Retorno:****

| Valor | Significado |
|--------|-------------------------------|
| `>= 0` | Descritor de arquivo (FD) |
| `-1` | Erro (arquivo não encontrado) |

****Comportamento:****

1. Tenta abrir como arquivo FATFS
2. Se falhar, tenta como dispositivo especial
3. Se sucesso, aloca um descritor
4. Retorna o FD

****Exemplo:****

```
```c
int fd = vfs_open("/boot.bat", O_RDONLY);
if (fd >= 0) {
 // Arquivo aberto com sucesso
}
```
```

5.2 vfs_read()

****Prototype:****

```
```c
int vfs_read(int fd, void *buffer, size_t size);
```
```

****Parâmetros:****

| Parâmetro | Tipo | Descrição |
|-----------|----------|----------------------------|
| `fd` | `int` | Descritor de arquivo |
| `buffer` | `void*` | Buffer para os dados lidos |
| `size` | `size_t` | Número de bytes a ler |

****Retorno:****

| Valor | Significado |
|-------|-------------|
| | |

| `> 0` | Número de bytes lidos |
| `0` | Fim de arquivo (EOF) |
| `-1` | Erro |

****Exemplo:****

```
```c
char buffer[128];
int bytes = vfs_read(fd, buffer, sizeof(buffer));
if (bytes > 0) {
 // Processa os dados lidos
}
```
```

5.3 vfs_write()

****Prototype:****

```
```c
int vfs_write(int fd, const void *buffer, size_t size);
```
```

****Parâmetros:****

| Parâmetro | Tipo | Descrição |
|-----------|---------------|----------------------------|
| `fd` | `int` | Descritor de arquivo |
| `buffer` | `const void*` | Dados a escrever |
| `size` | `size_t` | Número de bytes a escrever |

****Retorno:****

| Valor | Significado |
|-------|--------------------------|
| `> 0` | Número de bytes escritos |
| `-1` | Erro |

****Exemplo:****

```
```c
const char *msg = "Hello World\n";
vfs_write(1, msg, strlen(msg)); // Escreve no stdout
```
```

5.4 vfs_close()

****Prototype:****

```
```c
int vfs_close(int fd);
```
```

****Parâmetros:****

| Parâmetro | Tipo | Descrição |
|-----------|-------|----------------------|
| `fd` | `int` | Descritor de arquivo |

****Retorno:****

| Valor | Significado |
|-------|-------------|
| `0` | Sucesso |
| `-1` | Erro |

****Exemplo:****

```
```c
vfs_close(fd);
```
```

5.5 vfs_ioctl()

****Prototype:****

```
```c
int vfs_ioctl(int fd, int cmd, void *arg);
```
```

****Parâmetros:****

| Parâmetro | Tipo | Descrição |
|-----------|---------|----------------------|
| `fd` | `int` | Descritor de arquivo |
| `cmd` | `int` | Comando de controle |
| `arg` | `void*` | Argumento do comando |

****Retorno:****

| Valor | Significado |
|-------|-------------|
| `0` | Sucesso |
| `-1` | Erro |

****Exemplo:****

```
```c
// Limpa a tela
vfs_ioctl(1, TIOC_CLEAR, NULL);
```
```

...

5.6 vfs_lseek()

****Prototype:****

```
```c
size_t vfs_lseek(int fd, size_t offset, int whence);
```
```

****Parâmetros:****

| Parâmetro | Tipo | Descrição |
|-----------|----------|---------------------------------------|
| `fd` | `int` | Descritor de arquivo |
| `offset` | `size_t` | Deslocamento |
| `whence` | `int` | Referência (0=início, 1=atual, 2=fim) |

****Retorno:****

| Valor | Significado |
|--------|--------------|
| `>= 0` | Nova posição |
| `-1` | Erro |

****Exemplo:****

```
```c
// Vai para o início do arquivo
vfs_lseek(fd, 0, 0);

// Vai para o final do arquivo (se suportado)
vfs_lseek(fd, 0, 2);
```
```

6. SHELL E COMANDOS

6.1 Estrutura do Shell

```
```c
typedef struct {
 char *name;
 int (*func)(int argc, char **argv);
} Command;
```
```

6.2 Comandos Implementados

| Comando | Função | Descrição |
|------------|------------------|-------------------------------|
| `help` | `cmd_help()` | Lista comandos disponíveis |
| `echo` | `cmd_echo()` | Escreve texto no stdout |
| `cat` | `cmd_cat()` | Exibe conteúdo de arquivo |
| `ls` | `cmd_ls()` | Lista arquivos do diretório |
| `cd` | `cmd_cd()` | Muda diretório atual |
| `mkdir` | `cmd_mkdir()` | Cria um diretório |
| `reboot` | `cmd_reboot()` | Reinicia o sistema |
| `shutdown` | `cmd_shutdown()` | Desliga o sistema |
| `meminfo` | `cmd_meminfo()` | Mostra informações de memória |
| `clear` | `cmd_clear()` | Limpa a tela |

6.3 Exemplos de Uso

Exemplo 1: Lendo um arquivo

```
```bash
> cat /boot.bat
Script de inicialização do MeuSO
echo "Bem-vindo!"
meminfo
```
```

Exemplo 2: Escrevendo no terminal

```
```bash
> echo "Hello World"
Hello World
```
```

Exemplo 3: Lendo informações do sistema

```
```bash
> cat /proc/meminfo
MemTotal: 1024 KB
MemFree: 512 KB
MemUsed: 512 KB
MemAvail: 256 KB
```
```

Exemplo 4: Redirecionamento (futuro)

```
```bash
> echo "teste" > /tmp/arquivo.txt
> cat /tmp/arquivo.txt
teste
```
```

Exemplo 5: Dispositivos especiais

```
```bash
> cat /dev/null
(mostra nada)
> echo "texto" > /dev/null
(descarta a saída)
```
```

7. CÓDIGO FONTE COMPLETO

7.1 vfs.h

```
```c
#ifndef VFS_H
#define VFS_H

#include <stdint.h>
#include <stddef.h>
#include <fatfs/ff.h>

// =====
// TIPOS DE ARQUIVO
// =====
#define FILE_TYPE_REGULAR 0
#define FILE_TYPE_DEVICE 1
#define FILE_TYPE_PROC 2

// =====
// FLAGS PARA OPEN
// =====
#define O_RDONLY 0x01
#define O_WRONLY 0x02
#define O_RDWR 0x03
#define O_CREAT 0x04
#define O_TRUNC 0x08
#define O_APPEND 0x10

// =====
// COMANDOS IOCTL PARA TTY
// =====
#define TIOC_CLEAR 0x01
#define TIOC_GETXY 0x02
#define TIOC_SETXY 0x03
```

```

// =====
// ESTRUTURA FILE
// =====
typedef struct File {
 char *name;
 int type;
 size_t position;
 int ref_count;
 void *private_data;

 int (*open)(struct File *file, const char *path, int flags);
 int (*read)(struct File *file, void *buffer, size_t size);
 int (*write)(struct File *file, const void *buffer, size_t size);
 int (*close)(struct File *file);
 int (*ioctl)(struct File *file, int cmd, void *arg);
 size_t (*lseek)(struct File *file, size_t offset, int whence);
} File;

// =====
// ESTRUTURA DEVICE DRIVER
// =====
typedef struct {
 const char *path;
 int (*open)(File *file, const char *path, int flags);
 int (*read)(File *file, void *buffer, size_t size);
 int (*write)(File *file, const void *buffer, size_t size);
 int (*close)(File *file);
 int (*ioctl)(File *file, int cmd, void *arg);
 size_t (*lseek)(File *file, size_t offset, int whence);
} DeviceDriver;

// =====
// FUNÇÕES DO VFS
// =====
int vfs_open(const char *path, int flags);
int vfs_read(int fd, void *buffer, size_t size);
int vfs_write(int fd, const void *buffer, size_t size);
int vfs_close(int fd);
int vfs_ioctl(int fd, int cmd, void *arg);
size_t vfs_lseek(int fd, size_t offset, int whence);

// =====
// FUNÇÕES DE GERÊNCIA
// =====
int allocate_fd(File *file);

```

```

File *get_file_from_fd(int fd);
void free_fd(int fd);

// =====
// INICIALIZAÇÃO
// =====
void vfs_init(void);

#endif /* VFS_H */
```

---

## 7.2 vfs_fat.c

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "vfs.h"
#include "fileio.h" // SUA libfileio!

// =====
// ESTRUTURA PRIVADA PARA ARQUIVOS FATFS
// =====
typedef struct {
 FIL *fat_file;
 BYTE mode;
 FSIZE_t size;
} FatPrivate;

// =====
// FAT_OPEN
// =====
int fat_open(File *file, const char *path, int flags) {
 FIL *fat_file = (FIL*)malloc(sizeof(FIL));
 if (!fat_file) return -1;

 BYTE fat_mode = 0;
 if (flags & O_RDONLY) fat_mode |= FA_READ;
 if (flags & O_WRONLY) fat_mode |= FA_WRITE;
 if (flags & O_RDWR) fat_mode |= FA_READ | FA_WRITE;
 if (flags & O_CREAT) fat_mode |= FA_CREATE_ALWAYS;
 if (flags & O_APPEND) fat_mode |= FA_OPEN_APPEND;

 FRESULT result = fopen(fat_file, path, fat_mode);

```



```

 if (result != FR_OK) {
 free(fat_file);
 return -1;
 }

 FatPrivate *priv = (FatPrivate*)malloc(sizeof(FatPrivate));
 if (!priv) {
 fclose(fat_file);
 free(fat_file);
 return -1;
 }

 priv->fat_file = fat_file;
 priv->mode = fat_mode;
 priv->size = fsize(fat_file);

 file->private_data = priv;
 file->position = 0;
 file->read = fat_read;
 file->write = fat_write;
 file->close = fat_close;
 file->lseek = fat_lseek;

 return 0;
}

// =====
// FAT_READ
// =====
int fat_read(File *file, void *buffer, size_t size) {
 FatPrivate *priv = (FatPrivate*)file->private_data;
 if (!priv) return -1;

 UINT bytes_read;
 FRESULT result = fread(priv->fat_file, buffer, (UINT)size, &bytes_read);

 if (result != FR_OK) return -1;
 file->position += bytes_read;
 return (int)bytes_read;
}

// =====
// FAT_WRITE
// =====
int fat_write(File *file, const void *buffer, size_t size) {
 FatPrivate *priv = (FatPrivate*)file->private_data;

```

```

if (!priv) return -1;

UINT bytes_written;
FRESULT result = fwrite(priv->fat_file, buffer, (UINT)size, &bytes_written);

if (result != FR_OK) return -1;
file->position += bytes_written;

if (file->position > priv->size) {
 priv->size = file->position;
}

return (int)bytes_written;
}

// =====
// FAT_CLOSE
// =====
int fat_close(File *file) {
 FatPrivate *priv = (FatPrivate*)file->private_data;
 if (!priv) return -1;

 FRESULT result = fclose(priv->fat_file);
 free(priv->fat_file);
 free(priv);
 file->private_data = NULL;

 return (result == FR_OK) ? 0 : -1;
}

// =====
// FAT_LSEEK
// =====
size_t fat_lseek(File *file, size_t offset, int whence) {
 FatPrivate *priv = (FatPrivate*)file->private_data;
 if (!priv) return (size_t)-1;

 FSIZE_t new_pos;
 FSIZE_t current = ftell(priv->fat_file);

 switch (whence) {
 case 0: new_pos = (FSIZE_t)offset; break;
 case 1: new_pos = current + (FSIZE_t)offset; break;
 case 2: new_pos = priv->size + (FSIZE_t)offset; break;
 default: return (size_t)-1;
 }
}

```

```

 FRESULT result = flseek(priv->fat_file, new_pos);
 if (result != FR_OK) return (size_t)-1;

 file->position = (size_t)new_pos;
 return (size_t)new_pos;
}
...

```

---

## ## 7.3 vfs\_tty.c

```

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "vfs.h"

// =====
// FUNÇÕES DO SISTEMA
// =====
extern char getchar(void);
extern void putchar(char c);

// =====
// TTY_OPEN
// =====
int tty_open(File *file, const char *path, int flags) {
    file->private_data = NULL;
    file->position = 0;
    file->read = tty_read;
    file->write = tty_write;
    file->close = tty_close;
    file->ioctl = tty_ioctl;
    file->lseek = tty_lseek;
    return 0;
}

// =====
// TTY_READ
// =====
int tty_read(File *file, void *buffer, size_t size) {
    char *buf = (char*)buffer;
    size_t i = 0;

```

```

while (i < size) {
    char c = getchar();
    buf[i++] = c;
    putchar(c);

    if (c == '\r' || c == '\n') {
        putchar('\n');
        break;
    }
}

return (int)i;
}

// =====
// TTY_WRITE
// =====
int tty_write(File *file, const void *buffer, size_t size) {
    const char *data = (const char*)buffer;

    for (size_t i = 0; i < size; i++) {
        if (data[i] == '\n') {
            putchar('\r');
        }
        putchar(data[i]);
    }

    return (int)size;
}

// =====
// TTY_CLOSE
// =====
int tty_close(File *file) {
    return 0;
}

// =====
// TTY_IOCTL
// =====
int tty_ioctl(File *file, int cmd, void *arg) {
    switch (cmd) {
        case TIOC_CLEAR:
            printf("\033[2J\033[H");
            return 0;
        default:

```

```

        return -1;
    }
}

// =====
// TTY_LSEEK
// =====
size_t tty_lseek(File *file, size_t offset, int whence) {
    return (size_t)-1;
}
...

```

7.4 vfs_null.c

```

``c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "vfs.h"

// =====
// NULL_OPEN
// =====
int null_open(File *file, const char *path, int flags) {
    file->private_data = NULL;
    file->position = 0;
    file->read = null_read;
    file->write = null_write;
    file->close = null_close;
    file->iocctl = NULL;
    file->lseek = null_lseek;
    return 0;
}

// =====
// NULL_READ
// =====
int null_read(File *file, void *buffer, size_t size) {
    return 0; // EOF
}

// =====
// NULL_WRITE
// =====

```

```

int null_write(File *file, const void *buffer, size_t size) {
    return (int)size; // Descarta
}

// =====
// NULL_CLOSE
// =====
int null_close(File *file) {
    return 0;
}

// =====
// NULL_LSEEK
// =====
size_t null_lseek(File *file, size_t offset, int whence) {
    return offset;
}
...

```

7.5 vfs_zero.c

```

``c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "vfs.h"

// =====
// ZERO_OPEN
// =====
int zero_open(File *file, const char *path, int flags) {
    file->private_data = NULL;
    file->position = 0;
    file->read = zero_read;
    file->write = zero_write;
    file->close = zero_close;
    file->ioctl = NULL;
    file->lseek = zero_lseek;
    return 0;
}

// =====
// ZERO_READ
// =====

```

```

int zero_read(File *file, void *buffer, size_t size) {
    memset(buffer, 0, size);
    file->position += size;
    return (int)size;
}

// =====
// ZERO_WRITE
// =====
int zero_write(File *file, const void *buffer, size_t size) {
    return (int)size;
}

// =====
// ZERO_CLOSE
// =====
int zero_close(File *file) {
    return 0;
}

// =====
// ZERO_LSEEK
// =====
size_t zero_lseek(File *file, size_t offset, int whence) {
    switch (whence) {
        case 0: file->position = offset; break;
        case 1: file->position += offset; break;
        case 2: file->position = offset; break;
        default: return (size_t)-1;
    }
    return file->position;
}
...

```

7.6 vfs_proc.c

```

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "vfs.h"

// =====
// ESTRUTURA DE DADOS DO /proc

```

```

// =====
typedef struct {
 char *data;
 size_t size;
 size_t position;
} ProcData;

// =====
// FUNÇÕES AUXILIARES
// =====
int get_total_memory(void) { return 1024; }
int get_free_memory(void) { return 512; }
const char *get_cpu_name(void) { return "Motorola 68000"; }
int get_cpu_clock(void) { return 8; }

// =====
// PROC_READ
// =====
int proc_read(File *file, void *buffer, size_t size) {
 ProcData *data = (ProcData*)file->private_data;
 if (!data) return -1;

 size_t available = data->size - data->position;
 size_t to_read = (size < available) ? size : available;

 if (to_read == 0) return 0;

 memcpy(buffer, data->data + data->position, to_read);
 data->position += to_read;
 file->position = data->position;

 return (int)to_read;
}

// =====
// PROC_CLOSE
// =====
int proc_close(File *file) {
 ProcData *data = (ProcData*)file->private_data;
 if (data) {
 free(data->data);
 free(data);
 file->private_data = NULL;
 }
 return 0;
}

```



```

// =====
// PROC_LSEEK
// =====
size_t proc_lseek(File *file, size_t offset, int whence) {
 ProcData *data = (ProcData*)file->private_data;
 if (!data) return (size_t)-1;

 size_t new_pos;
 switch (whence) {
 case 0: new_pos = offset; break;
 case 1: new_pos = data->position + offset; break;
 case 2: new_pos = data->size + offset; break;
 default: return (size_t)-1;
 }

 if (new_pos > data->size) new_pos = data->size;
 data->position = new_pos;
 file->position = new_pos;
 return new_pos;
}

// =====
// PROC_MEMINFO_OPEN
// =====
int proc_meminfo_open(File *file, const char *path, int flags) {
 ProcData *data = (ProcData*)malloc(sizeof(ProcData));
 if (!data) return -1;

 char buffer[256];
 sprintf(buffer,
 "MemTotal: %d KB\n"
 "MemFree: %d KB\n"
 "MemUsed: %d KB\n"
 "MemAvail: %d KB\n",
 get_total_memory(),
 get_free_memory(),
 get_total_memory() - get_free_memory(),
 get_free_memory() / 2
);

 data->data = strdup(buffer);
 data->size = strlen(buffer);
 data->position = 0;

 file->private_data = data;

```

```

file->position = 0;
file->read = proc_read;
file->write = NULL;
file->close = proc_close;
file->ioctl = NULL;
file->lseek = proc_lseek;

return 0;
}

// =====
// PROC_CPUINFO_OPEN
// =====
int proc_cpuinfo_open(File *file, const char *path, int flags) {
 ProcData *data = (ProcData*)malloc(sizeof(ProcData));
 if (!data) return -1;

 char buffer[256];
 sprintf(buffer,
 "CPU: %s\n"
 "Clock: %d MHz\n"
 "Features: FPU (sim)\n"
 "Cores: 1\n"
 "Family: m68k\n",
 get_cpu_name(),
 get_cpu_clock()
);

 data->data = strdup(buffer);
 data->size = strlen(buffer);
 data->position = 0;

 file->private_data = data;
 file->position = 0;
 file->read = proc_read;
 file->write = NULL;
 file->close = proc_close;
 file->ioctl = NULL;
 file->lseek = proc_lseek;

 return 0;
}

// =====
// PROC_UPTIME_OPEN
// =====

```

```

int proc_uptime_open(File *file, const char *path, int flags) {
 ProcData *data = (ProcData*)malloc(sizeof(ProcData));
 if (!data) return -1;

 static int uptime = 0;
 uptime += 10;

 char buffer[64];
 sprintf(buffer, "%d seconds\n", uptime);

 data->data = strdup(buffer);
 data->size = strlen(buffer);
 data->position = 0;

 file->private_data = data;
 file->position = 0;
 file->read = proc_read;
 file->write = NULL;
 file->close = proc_close;
 file->ioctl = NULL;
 file->lseek = proc_lseek;

 return 0;
}
...

```

---

## 7.7 vfs\_drivers.c

```c

#include "vfs.h"

```

// =====
// DECLARAÇÕES EXTERNAS DOS DRIVERS
// =====
extern int tty_open(File *file, const char *path, int flags);
extern int tty_read(File *file, void *buffer, size_t size);
extern int tty_write(File *file, const void *buffer, size_t size);
extern int tty_close(File *file);
extern int tty_ioctl(File *file, int cmd, void *arg);
extern size_t tty_lseek(File *file, size_t offset, int whence);

extern int null_open(File *file, const char *path, int flags);
extern int null_read(File *file, void *buffer, size_t size);
extern int null_write(File *file, const void *buffer, size_t size);

```

```
extern int null_close(File *file);
extern size_t null_lseek(File *file, size_t offset, int whence);
```

```
extern int zero_open(File *file, const char *path, int flags);
extern int zero_read(File *file, void *buffer, size_t size);
extern int zero_write(File *file, const void *buffer, size_t size);
extern int zero_close(File *file);
extern size_t zero_lseek(File *file, size_t offset, int whence);
```

```
extern int proc_meminfo_open(File *file, const char *path, int flags);
extern int proc_cpuinfo_open(File *file, const char *path, int flags);
extern int proc_uptime_open(File *file, const char *path, int flags);
extern int proc_read(File *file, void *buffer, size_t size);
extern int proc_close(File *file);
extern size_t proc_lseek(File *file, size_t offset, int whence);
```

```
// =====
// TABELA DE DRIVERS
// =====
DeviceDriver drivers[] = {
    {"/dev/tty", tty_open,      tty_read,  tty_write,  tty_close,  tty_ioctl,
    tty_lseek},
    {"/dev/null", null_open,    null_read, null_write, null_close, NULL,
    null_lseek},
    {"/dev/zero", zero_open,    zero_read, zero_write, zero_close, NULL,
    zero_lseek},
    {"/proc/meminfo", proc_meminfo_open, proc_read, NULL,    proc_close,
    NULL,    proc_lseek},
    {"/proc/cpuinfo", proc_cpuinfo_open, proc_read, NULL,    proc_close, NULL,
    proc_lseek},
    {"/proc/uptime", proc_uptime_open, proc_read, NULL,    proc_close, NULL,
    proc_lseek},
    {NULL, NULL, NULL, NULL, NULL, NULL, NULL}
};
---
```

```
## 7.8 vfs.c
```

```
``c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "vfs.h"
```

```

// =====
// TABELA DE DESCRITORES GLOBAL
// =====
static File *fd_table[256];
static int fd_count = 0;

extern DeviceDriver drivers[];

// =====
// ALLOCATE_FD
// =====
int allocate_fd(File *file) {
    for (int i = 3; i < 256; i++) {
        if (fd_table[i] == NULL) {
            fd_table[i] = file;
            fd_count++;
            return i;
        }
    }
    return -1;
}

// =====
// GET_FILE_FROM_FD
// =====
File *get_file_from_fd(int fd) {
    if (fd < 0 || fd >= 256) return NULL;
    return fd_table[fd];
}

// =====
// FREE_FD
// =====
void free_fd(int fd) {
    if (fd >= 0 && fd < 256) {
        fd_table[fd] = NULL;
        fd_count--;
    }
}

// =====
// VFS_OPEN
// =====
int vfs_open(const char *path, int flags) {
    File *file = (File*)malloc(sizeof(File));
    if (!file) return -1;

```

```
memset(file, 0, sizeof(File));
file->name = strdup(path);
file->type = FILE_TYPE_REGULAR;
file->position = 0;
file->ref_count = 1;
```

```
extern int fat_open(File *file, const char *path, int flags);
if (fat_open(file, path, flags) == 0) {
    int fd = allocate_fd(file);
    if (fd >= 0) return fd;
    free(file->name);
    free(file);
    return -1;
}
```

```
for (int i = 0; drivers[i].path != NULL; i++) {
    if (strcmp(path, drivers[i].path) == 0) {
        file->type = FILE_TYPE_DEVICE;
        file->read = drivers[i].read;
        file->write = drivers[i].write;
        file->close = drivers[i].close;
        file->ioctl = drivers[i].ioctl;
        file->lseek = drivers[i].lseek;

        if (drivers[i].open(file, path, flags) == 0) {
            int fd = allocate_fd(file);
            if (fd >= 0) return fd;
        }
        break;
    }
}
```

```
free(file->name);
free(file);
return -1;
}
```

```
// =====
// VFS_READ
// =====
int vfs_read(int fd, void *buffer, size_t size) {
    File *file = get_file_from_fd(fd);
    if (!file) return -1;
    if (file->read) return file->read(file, buffer, size);
    return -1;
}
```

```

}

// =====
// VFS_WRITE
// =====
int vfs_write(int fd, const void *buffer, size_t size) {
    File *file = get_file_from_fd(fd);
    if (!file) return -1;
    if (file->write) return file->write(file, buffer, size);
    return -1;
}

// =====
// VFS_CLOSE
// =====
int vfs_close(int fd) {
    File *file = get_file_from_fd(fd);
    if (!file) return -1;

    int result = 0;
    if (file->close) result = file->close(file);

    free(file->name);
    free(file);
    free_fd(fd);
    return result;
}

// =====
// VFS_IOCTL
// =====
int vfs_ioctl(int fd, int cmd, void *arg) {
    File *file = get_file_from_fd(fd);
    if (!file) return -1;
    if (file->ioctl) return file->ioctl(file, cmd, arg);
    return -1;
}

// =====
// VFS_LSEEK
// =====
size_t vfs_lseek(int fd, size_t offset, int whence) {
    File *file = get_file_from_fd(fd);
    if (!file) return (size_t)-1;

    if (file->lseek) return file->lseek(file, offset, whence);

```

```

switch (whence) {
    case 0: file->position = offset; break;
    case 1: file->position += offset; break;
    case 2: return (size_t)-1;
    default: return (size_t)-1;
}
return file->position;
}

// =====
// VFS_INIT
// =====
void vfs_init(void) {
    memset(fd_table, 0, sizeof(fd_table));
    fd_count = 0;

    int fd0 = vfs_open("/dev/tty", O_RDWR);
    int fd1 = vfs_open("/dev/tty", O_RDWR);
    int fd2 = vfs_open("/dev/tty", O_RDWR);

    if (fd0 != 0) {
        fd_table[0] = fd_table[fd0];
        fd_table[fd0] = NULL;
        fd_count--;
    }
    if (fd1 != 1) {
        fd_table[1] = fd_table[fd1];
        fd_table[fd1] = NULL;
        fd_count--;
    }
    if (fd2 != 2) {
        fd_table[2] = fd_table[fd2];
        fd_table[fd2] = NULL;
        fd_count--;
    }

    printf("VFS inicializado: stdin=0, stdout=1, stderr=2\n");
}
...

---

## 7.9 shell.c

```c

```



```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "vfs.h"

// =====
// PROTÓTIPOS DOS COMANDOS
// =====
int cmd_help(int argc, char **argv);
int cmd_echo(int argc, char **argv);
int cmd_cat(int argc, char **argv);
int cmd_ls(int argc, char **argv);
int cmd_cd(int argc, char **argv);
int cmd_mkdir(int argc, char **argv);
int cmd_reboot(int argc, char **argv);
int cmd_shutdown(int argc, char **argv);
int cmd_meminfo(int argc, char **argv);
int cmd_clear(int argc, char **argv);

// =====
// TABELA DE COMANDOS
// =====
Command commands[] = {
 {"help", cmd_help},
 {"echo", cmd_echo},
 {"cat", cmd_cat},
 {"ls", cmd_ls},
 {"cd", cmd_cd},
 {"mkdir", cmd_mkdir},
 {"reboot", cmd_reboot},
 {"shutdown", cmd_shutdown},
 {"meminfo", cmd_meminfo},
 {"clear", cmd_clear},
 {NULL, NULL}
};

// =====
// GETS_LINE
// =====
char *gets_line(char *buffer, int size) {
 int i = 0;
 char c;

 while (i < size - 1) {
 c = getchar();
 }

```

```

 if (c == '\r' || c == '\n') {
 buffer[i] = '\0';
 putchar("\n");
 return buffer;
 }
 else if (c == '\b' || c == 0x7F) {
 if (i > 0) {
 i--;
 putchar("\b");
 putchar(' ');
 putchar("\b");
 }
 }
 else if (c >= 0x20 && c < 0x7F) {
 buffer[i++] = c;
 putchar(c);
 }
}
buffer[i] = '\0';
return buffer;
}

// =====
// EXECUTE_LINE
// =====
int execute_line(char *line) {
 char *args[16];
 int argc = 0;
 char *token;
 char line_copy[256];

 while (*line == ' ') line++;
 line[strcspn(line, "\n")] = 0;
 line[strcspn(line, "\r")] = 0;

 if (line[0] == '\0' || line[0] == '#') return 0;

 strcpy(line_copy, line);

 token = strtok(line_copy, " ");
 while (token != NULL && argc < 16) {
 args[argc++] = token;
 token = strtok(NULL, " ");
 }

 if (argc == 0) return 0;

```

```

 for (int i = 0; commands[i].name != NULL; i++) {
 if (strcmp(commands[i].name, args[0]) == 0) {
 return commands[i].func(argc, args);
 }
 }

 vfs_write(1, "Comando desconhecido: ", 23);
 vfs_write(1, args[0], strlen(args[0]));
 vfs_write(1, "\n", 1);
 return -1;
}

// =====
// COMANDOS
// =====

int cmd_help(int argc, char **argv) {
 vfs_write(1, "Comandos disponíveis:\n", 23);
 for (int i = 0; commands[i].name != NULL; i++) {
 vfs_write(1, " ", 2);
 vfs_write(1, commands[i].name, strlen(commands[i].name));
 vfs_write(1, "\n", 1);
 }
 return 0;
}

int cmd_echo(int argc, char **argv) {
 for (int i = 1; i < argc; i++) {
 vfs_write(1, argv[i], strlen(argv[i]));
 if (i < argc - 1) vfs_write(1, " ", 1);
 }
 vfs_write(1, "\n", 1);
 return 0;
}

int cmd_cat(int argc, char **argv) {
 if (argc < 2) {
 vfs_write(1, "Uso: cat <arquivo>\n", 20);
 return -1;
 }

 int fd = vfs_open(argv[1], O_RDONLY);
 if (fd < 0) {
 vfs_write(1, "Erro: não foi possível abrir ", 29);
 vfs_write(1, argv[1], strlen(argv[1]));
 }
}

```

```

 vfs_write(1, "\n", 1);
 return -1;
}

char buffer[128];
int bytes;
while ((bytes = vfs_read(fd, buffer, sizeof(buffer))) > 0) {
 vfs_write(1, buffer, bytes);
}

vfs_close(fd);
return 0;
}

int cmd_ls(int argc, char **argv) {
 DIR dir;
 FILINFO fno;
 const char *path = (argc > 1) ? argv[1] : "/";

 if (fopendir(&dir, path) != FR_OK) {
 vfs_write(1, "Erro: não foi possível listar ", 30);
 vfs_write(1, path, strlen(path));
 vfs_write(1, "\n", 1);
 return -1;
 }

 while (freaddir(&dir, &fno) == FR_OK && fno.fname[0] != 0) {
 vfs_write(1, fno.fname, strlen(fno.fname));
 if (fno.fattrib & AM_DIR) {
 vfs_write(1, "/", 1);
 }
 vfs_write(1, " ", 2);
 }
 vfs_write(1, "\n", 1);

 fclosedir(&dir);
 return 0;
}

int cmd_cd(int argc, char **argv) {
 if (argc < 2) {
 vfs_write(1, "cd: falta diretório\n", 21);
 return -1;
 }
 vfs_write(1, "cd: mudando para ", 18);
 vfs_write(1, argv[1], strlen(argv[1]));
}

```

```

 vfs_write(1, "\n", 1);
 return 0;
}

int cmd_mkdir(int argc, char **argv) {
 if (argc < 2) {
 vfs_write(1, "mkdir: falta diretório\n", 24);
 return -1;
 }

 FRESULT result = fmkdir(argv[1]);
 if (result != FR_OK) {
 vfs_write(1, "Erro ao criar diretório\n", 25);
 return -1;
 }

 vfs_write(1, "mkdir: criado ", 15);
 vfs_write(1, argv[1], strlen(argv[1]));
 vfs_write(1, "\n", 1);
 return 0;
}

int cmd_reboot(int argc, char **argv) {
 vfs_write(1, "Reiniciando sistema...\n", 24);
 return 0;
}

int cmd_shutdown(int argc, char **argv) {
 vfs_write(1, "Desligando sistema...\n", 23);
 return 0;
}

int cmd_meminfo(int argc, char **argv) {
 int fd = vfs_open("/proc/meminfo", O_RDONLY);
 if (fd < 0) {
 vfs_write(1, "Erro: /proc/meminfo não disponível\n", 36);
 return -1;
 }

 char buffer[128];
 int bytes;
 while ((bytes = vfs_read(fd, buffer, sizeof(buffer))) > 0) {
 vfs_write(1, buffer, bytes);
 }

 vfs_close(fd);
}

```

```

 return 0;
}

int cmd_clear(int argc, char **argv) {
 int fd = vfs_open("/dev/tty", O_RDWR);
 if (fd >= 0) {
 vfs_ioctl(fd, TIOC_CLEAR, NULL);
 vfs_close(fd);
 }
 return 0;
}

// =====
// RUN_BOOT_SCRIPT
// =====
void run_boot_script(void) {
 int fd = vfs_open("/boot.bat", O_RDONLY);
 if (fd < 0) {
 vfs_write(1, "Nenhum script de boot encontrado\n", 34);
 return;
 }

 vfs_write(1, "\n=== Executando /boot.bat ===\n", 31);

 char line[256];
 int pos = 0;
 char c;
 int bytes;

 while ((bytes = vfs_read(fd, &c, 1)) > 0) {
 if (c == '\n' || c == '\r') {
 if (pos > 0) {
 line[pos] = '\0';
 pos = 0;

 if (line[0] != '#') {
 vfs_write(1, "[boot] ", 7);
 vfs_write(1, line, strlen(line));
 vfs_write(1, "\n", 1);
 execute_line(line);
 }
 }
 continue;
 }
 if (pos < sizeof(line) - 1) {
 line[pos++] = c;
 }
 }
}

```

```

 }
}

if (pos > 0) {
 line[pos] = '\0';
 if (line[0] != '#') {
 vfs_write(1, "[boot] ", 7);
 vfs_write(1, line, strlen(line));
 vfs_write(1, "\n", 1);
 execute_line(line);
 }
}

vfs_close(fd);
vfs_write(1, "=== Script de boot finalizado ===\n\n", 36);
}

// =====
// SHELL_LOOP
// =====
void shell_loop(void) {
 char line[256];

 run_boot_script();

 vfs_write(1, "--- Shell do MeuSO ---\n", 24);
 vfs_write(1, "Digite 'help' para comandos\n\n", 30);

 while (1) {
 vfs_write(1, "> ", 2);
 gets_line(line, sizeof(line));
 execute_line(line);
 }
}

// =====
// MAIN
// =====
int main(void) {
 vfs_init();
 shell_loop();
 return 0;
}
...

```

## ## 7.10 Makefile

```
``makefile
=====
MAKEFILE PARA MEUSO - VFS
=====

CC = m68k-elf-gcc
CFLAGS = -Wall -Os -I.././../include -I. \
 -ffreestanding -nostdlib \
 -D __68000__ -D ALIGN_MEMORY
LDFLAGS = -nostdlib -T link.ld

VFS_SRCS = vfs.c vfs_fat.c vfs_tty.c vfs_null.c vfs_zero.c \
 vfs_proc.c vfs_drivers.c shell.c
VFS_OBJS = $(VFS_SRCS:.c=.o)

LIBFILEIO = .././../lib/libfileio.a

TARGET = shell.elf

all: $(TARGET)

$(TARGET): $(VFS_OBJS) $(LIBFILEIO)
 $(CC) $(LDFLAGS) -o $@ $^

%.o: %.c
 $(CC) $(CFLAGS) -c $< -o $@

clean:
 rm -f $(VFS_OBJS) $(TARGET)

install: $(TARGET)
 cp $(TARGET) /path/to/your/rootfs/bin/shell

.PHONY: all clean install
``
```

---

## # 8. COMPILAÇÃO E INSTALAÇÃO

### ## 8.1 Pré-requisitos

Item	Descrição
------	-----------



```
|-----|-----|
| Compilador | `m68k-elf-gcc` |
| Bibliotecas | `libfileio.a` (suas syscalls) |
| Headers | FATFS (`ff.h`, `fileio.h`) |
| Sistema | MeuSO com traps implementados |
```

## ## 8.2 Compilação

```
```bash
# 1. Limpar compilações anteriores
make clean

# 2. Compilar
make

# 3. Verificar resultado
ls -la shell.elf
```
```

## ## 8.3 Instalação

```
```bash
# Instalar no sistema
make install

# Ou manualmente
cp shell.elf /path/to/rootfs/bin/shell
```
```

## ## 8.4 Estrutura de Diretórios

```
...
/
├── boot.bat # Script de inicialização
├── bin/
│ └── shell.elf # Shell com VFS
├── dev/
│ ├── tty # Terminal
│ ├── null # Dispositivo nulo
│ └── zero # Gerador de zeros
├── proc/
│ ├── meminfo # Informações de memória
│ ├── cpuinfo # Informações da CPU
│ └── uptime # Tempo de atividade
├── tmp/ # Arquivos temporários
└── var/ # Dados variáveis
```

...

---

## # 9. TESTES E VALIDAÇÃO

### ## 9.1 Teste de Arquivos

**\*\*Objetivo:\*\*** Verificar leitura/escrita de arquivos reais

```
```bash
```

```
# Criar arquivo
```

```
echo "Hello World" > /tmp/test.txt
```

```
# Ler arquivo
```

```
cat /tmp/test.txt
```

```
# Saída esperada: Hello World
```

```
# Verificar tamanho
```

```
ls -l /tmp/test.txt
```

```
```
```

**\*\*Resultado esperado:\*\***  Sucesso

---

### ## 9.2 Teste de Dispositivos

**\*\*Objetivo:\*\*** Verificar dispositivos especiais

```
```bash
```

```
# Testar TTY
```

```
echo "Teste TTY" > /dev/tty
```

```
# Saída esperada: Teste TTY
```

```
# Testar NULL
```

```
echo "Descartado" > /dev/null
```

```
# (sem saída)
```

```
# Testar ZERO
```

```
dd if=/dev/zero of=zeros.bin bs=10 count=1
```

```
# Cria arquivo com 10 bytes de zeros
```

```
```
```

**\*\*Resultado esperado:\*\***  Sucesso

---

### ## 9.3 Teste de /proc

**\*\*Objetivo:\*\*** Verificar informações do sistema

```bash

Memória

cat /proc/meminfo

Saída esperada:

MemTotal: 1024 KB

MemFree: 512 KB

CPU

cat /proc/cpuinfo

Saída esperada:

CPU: Motorola 68000

Clock: 8 MHz

Uptime

cat /proc/uptime

Saída esperada: X seconds

```

**\*\*Resultado esperado:\*\***  Sucesso

---

### ## 9.4 Teste de Boot Script

**\*\*Objetivo:\*\*** Verificar execução do script de boot

**\*\*Arquivo `/boot.bat`:\*\***

```batch

Script de inicialização

echo "MeuSO iniciando..."

meminfo

echo "Sistema pronto!"

```

**\*\*Saída esperada:\*\***

```

=== Executando /boot.bat ===

[boot] echo "MeuSO iniciando..."

MeuSO iniciando...

[boot] meminfo

```
MemTotal: 1024 KB
MemFree: 512 KB
[boot] echo "Sistema pronto!"
Sistema pronto!
=== Script de boot finalizado ===
...
```

****Resultado esperado:****  Sucesso

10. REFERÊNCIAS

10.1 Documentação

| Recurso | Descrição |
|-----------------|---|
| Linux VFS | https://www.kernel.org/doc/html/latest/filesystems/vfs.html |
| FatFS | http://elm-chan.org/fsw/ff/00index_e.html |
| m68k Assembly | https://en.wikipedia.org/wiki/Motorola_68000 |
| UNIX Philosophy | https://en.wikipedia.org/wiki/Everything_is_a_file |

10.2 Código Fonte do MeuSO

| Arquivo | Descrição |
|------------|-------------------------------------|
| `fileio.h` | Header da libfileio (suas syscalls) |
| `fileio.c` | Implementação das syscalls |
| `trap12.S` | Handler da trap #12 |
| `trap12.c` | Interface C para FATFS |

10.3 Licença

Este projeto é parte do sistema operacional MeuSO. Todos os direitos reservados.

APÊNDICE A: MAPEAMENTO DE FLAGS

| VFS Flag | Valor | FATFS Flag | Descrição |
|------------|-------|------------------------|----------------------|
| `O_RDONLY` | 0x01 | `FA_READ` | Somente leitura |
| `O_WRONLY` | 0x02 | `FA_WRITE` | Somente escrita |
| `O_RDWR` | 0x03 | `FA_READ` `FA_WRITE` | Leitura e escrita |
| `O_CREAT` | 0x04 | `FA_CREATE_ALWAYS` | Criar se não existir |
| `O_TRUNC` | 0x08 | `FA_CREATE_ALWAYS` | Truncar se existir |

```
| `O_APPEND` | 0x10 | `FA_OPEN_APPEND` | Escrever no final |
```

APÊNDICE B: DESCRITORES PADRÃO

```
FD	Nome	Dispositivo	Descrição
0	stdin	`/dev/tty`	Entrada padrão (teclado)
1	stdout	`/dev/tty`	Saída padrão (vídeo)
2	stderr	`/dev/tty`	Saída de erro (vídeo)
```

APÊNDICE C: COMANDOS IOCTL

```
Comando	Valor	Descrição
`TIOC_CLEAR`	0x01	Limpa a tela
`TIOC_GETXY`	0x02	Obtém posição do cursor
`TIOC_SETXY`	0x03	Define posição do cursor
```

APÊNDICE D: EXEMPLO DE SESSÃO

...

```
=== Executando /boot.bat ===  
[boot] echo "MeuSO v1.0 iniciando..."  
MeuSO v1.0 iniciando...  
[boot] meminfo  
MemTotal: 1024 KB  
MemFree: 512 KB  
MemUsed: 512 KB  
MemAvail: 256 KB  
[boot] echo "Sistema pronto!"  
Sistema pronto!  
=== Script de boot finalizado ===
```

```
--- Shell do MeuSO ---  
Digite 'help' para comandos
```

```
> help  
Comandos disponíveis:  
  help  
  echo
```

```
cat
ls
cd
mkdir
reboot
shutdown
meminfo
clear
```

```
> echo "Testando VFS"
Testando VFS
```

```
> cat /proc/cpuinfo
CPU:      Motorola 68000
Clock:    8 MHz
Features: FPU (sim)
Cores:    1
Family:   m68k
```

```
> echo "Arquivo de teste" > /tmp/test.txt
> cat /tmp/test.txt
Arquivo de teste
```

```
> cat /dev/null
> echo "Descartado" > /dev/null
> clear
(tela limpa)
```

```
> shutdown
Desligando sistema...
^^^
```

```
# FIM DO DOCUMENTO
```

```
**MeuSO - Um sistema operacional para m68k**
**Versão 1.0**
**Agosto 2026**
```

Agora você tem ****todo o conteúdo formatado**** para colar no LibreOffice! Basta:

1. Copiar o texto acima
2. Colar no LibreOffice Writer
3. Aplicar os estilos:
 - **Título 1** para os capítulos (1, 2, 3...)
 - **Título 2** para subcapítulos (1.1, 2.1...)
 - **Código** com fonte monoespaçada (Courier New)
 - **Tabelas** com bordas
4. Gerar o sumário automaticamente (Inserir → Índice e Sumário)
5. Salvar como `.odt` e exportar como `.pdf`

Boa sorte com seu MeuSO! 🚀